

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 04: Range Queries

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
6 de noviembre de 2026

Índice

1. Resumen **1**
1.1. Propósito y alcance del estudio . . . 1
1.2. Estructuras y algoritmos estudiados . . . 1

2. Análisis teórico **2**
2.1. Sparse Table . . . 2
2.2. Segment Tree . . . 3
2.3. Comparación de complejidades . . . 3

3. Implementación **4**
3.1. Sparse Table . . . 4
3.2. Segment Tree . . . 4

4. Metodología experimental **5**
4.1. Características de la máquina . . . 5
4.2. Datos de entrada . . . 5
4.3. Experimentos e hipótesis . . . 5
4.3.1. Construcción inicial en el escenario estático . . . 6
4.3.2. Consultas en el escenario estático . . . 6
4.3.3. Mantenimiento en el escenario dinámico . . . 6
4.3.4. Consultas en el escenario dinámico . . . 7
4.3.5. Análisis del espacio utilizado . . . 7

Referencias **8**

1. Resumen

1.1. Propósito y alcance del estudio

Este estudio compara dos estructuras de datos para resolver el problema de *Range Minimum Query* (RMQ): **Sparse Table** y **Segment Tree**. Dado un arreglo de enteros A, una consulta sobre el rango inclusivo $A[L..R]$ busca obtener el **índice del elemento mínimo**, no solamente su valor. Se consideran un **escenario estático**, en el que el arreglo permanece fijo, y un **escenario dinámico**, en el que sus elementos reciben modificaciones puntuales.

El diseño experimental contempla cuatro comparaciones:

- **Construcción en el escenario estático:** medir el tiempo necesario para construir ambas estructuras a partir del mismo arreglo, considerando diferentes cantidades de elementos.
- **Consultas en el escenario estático:** medir el tiempo de respuesta sobre las estructuras ya construidas, utilizando rangos aleatorios de distintas longitudes y arreglos de diferentes tamaños.
- **Mantenimiento en el escenario dinámico:** comparar el tiempo de una actualización puntual en Segment Tree con el tiempo de reconstruir Sparse Table después del mismo cambio. Esta comparación representa el costo de mantener respuestas vigentes, no dos operaciones de actualización equivalentes.
- **Consultas en el escenario dinámico:** medir el tiempo de respuesta después de aplicar los cambios y mantener ambas estructuras actualizadas, separándolo del costo de mantenimiento.

En cada comparación se utilizarán las mismas entradas y consultas para ambas estructuras. Además, se realizará un **análisis del espacio utilizado**, considerando las asignaciones de memoria de los constructores y las reservas temporales relevantes durante la construcción o reconstrucción. Los resultados se interpretarán junto con las complejidades teóricas para discutir las ventajas y limitaciones de cada alternativa. El estudio también abordará su adaptación a otras operaciones binarias y las propiedades algebraicas que permiten conservar sus garantías.

1.2. Estructuras y algoritmos estudiados

- **Sparse Table:** preprocesa el arreglo almacenando resultados para bloques de longitud potencia de dos. Para RMQ permite responder consultas en tiempo constante mediante la combinación de dos bloques que pueden solaparse. La implementación estudiada no ofrece actualizaciones, por lo que en el escenario dinámico se reconstruirá después de cada modificación.
- **Segment Tree:** organiza los mínimos de intervalos en una estructura jerárquica. Responde consultas y permite actualizar un elemento en tiempo logarítmico, recalculando los nodos afectados sin reconstruir toda la estructura.

Las implementaciones incorporadas proceden del repositorio oficial del curso, con autoría original de LELE [1, 2].

2. Análisis teórico

Sea A un arreglo no vacío de n enteros. Una consulta RMQ sobre $A[L..R]$, con $0 \le L \le R < n$, devuelve el índice de su elemento mínimo. Denotamos por $k = R - L + 1$ la longitud del rango y resolvemos los empates con el **menor índice**. Para ello, cada candidato se representa mediante el par $(A[i], i)$: el mínimo lexicográfico compara primero el valor y luego el índice. Las implementaciones reciben intervalos semiabiertos $[L, R+1)$; convertir los extremos y extraer el índice de la respuesta cuesta $\Theta(1)$.

Usamos $O(\cdot)$ para cotas superiores y $\Theta(\cdot)$ para cotas ajustadas. Suponemos un modelo de palabras de máquina: valores, índices y pares ocupan espacio constante, y acceder, copiar y comparar un par cuesta $\Theta(1)$. El espacio de la estructura incluye sus copias internas, pero excluye el arreglo externo de entrada; devolver un índice ocupa $\Theta(1)$. Las expresiones con $\log n$, en base dos, se presentan para $n \ge 2$; con un solo elemento las operaciones RMQ son constantes. El análisis se deriva del código del curso y de las descripciones de ambas estructuras [1, 2, 3, 4].

2.1. Sparse Table

Sparse Table almacena respuestas para bloques de longitud potencia de dos. Si $S_j[i]$ es el mínimo lexicográfico del bloque que comienza en i y tiene 2^j elementos, su construcción utiliza

$$S_0[i] = (A[i], i), \qquad S_j[i] = \min_{\text{lex}}\big(S_{j-1}[i], S_{j-1}[i + 2^{j-1}]\big).$$

La implementación reserva $\ell = \lfloor \log_2 n \rfloor + 1$ filas completas de n elementos, aunque algunas posiciones no correspondan a bloques válidos [1]. Para una consulta, toma $b = \lfloor \log_2 k \rfloor$ y combina

$$\min_{\text{lex}}\big(S_b[L], S_b[R - 2^b + 1]\big).$$

Los dos bloques cubren el rango y pueden solaparse. La duplicación de su intersección no cambia el mínimo, por la idempotencia de esta operación [3].

- **Preparación (construcción):** $\Theta(n \log n)$, tanto en mejor como en peor caso. Inicializar las filas ya requiere $\Theta(n\ell)$ trabajo. Además, si $m = \lfloor \log_2 n \rfloor$, se realizan $\sum_{j=1}^{m}(n - 2^j + 1) = m(n+1) - 2^{m+1} + 2$ combinaciones; cada una cuesta tiempo constante.
- **Mejor caso de consulta RMQ:** $\Theta(1)$.
- **Peor caso de consulta RMQ:** $\Theta(1)$. El método `query_idempotent` realiza dos accesos y una combinación. Calcula el nivel con `std::bit_width`, que permite obtener $\lfloor \log_2 k \rfloor$ sobre una palabra de ancho fijo sin recorrer el rango [3, 6].
- **Consulta general:** el método distinto `query` descompone k en potencias de dos y combina bloques disjuntos. Procesa tantos bloques como bits uno tiene k: su costo es $\Theta(\text{popcount}(k))$, con mejor caso $\Theta(1)$ y cota $O(\log(k+1))$. No es la variante utilizada para medir RMQ.
- **Mantenimiento tras un cambio:** la implementación no ofrece `update`. En el escenario dinámico se reconstruye la tabla, con costo $\Theta(n \log n)$ por modificación, independientemente de su posición. Es una estrategia de mantenimiento del estudio, no una operación de actualización nativa ni una imposibilidad de diseñar otras variantes.
- **Memoria:** $\Theta(n \log n)$ persistente y $\Theta(1)$ auxiliar por consulta. Para $s = \text{sizeof}(T)$, las filas contienen $n\ell$ objetos de tipo T, cuyo tamaño lógico de datos es $n\ell s$ bytes, además de los metadatos de los vectores y del combinador. La expresión que crea las filas utiliza también un vector temporal de n elementos: $\Theta(n)$ espacio temporal, sin cambiar el orden $\Theta(n \log n)$ del pico total de construcción.

El tamaño lógico no debe confundirse con los bytes efectivamente reservados: estos dependen de las capacidades de los vectores y de la biblioteca usada. Si durante una reconstrucción coexisten la tabla anterior y la nueva, el pico puede aumentar, pero conserva el mismo orden asintótico.

2.2. Segment Tree

Segment Tree mantiene mínimos de grupos de elementos en una jerarquía binaria [4]. La versión del curso es iterativa: reserva un vector de $2n$ posiciones, coloca las hojas en $[n, 2n)$ y calcula los nodos internos combinando sus dos hijos. No redondea n a una potencia de dos [2].

- **Preparación (construcción):** $\Theta(n)$, tanto en mejor como en peor caso. Inicializa $2n$ posiciones, copia n hojas y calcula exactamente $n-1$ mínimos de hijos. No realiza n actualizaciones independientes, lo que costaría $O(n \log n)$.
- **Mejor caso de consulta:** $\Theta(1)$, por ejemplo al consultar un único elemento. En ese caso el intervalo se consume en una iteración del recorrido ascendente.
- **Peor caso de consulta:** $\Theta(\log n)$. Los dos extremos ascienden dividiéndose por dos y se incorporan como máximo dos nodos por nivel. Existen rangos que requieren recorrer $\Theta(\log n)$ niveles. Más precisamente, para longitud k el bucle queda acotado por $O(\log(k+1))$, porque la cantidad de posiciones pendientes se reduce al menos a la mitad en cada paso. Por ello, mantener k fijo no obliga a observar crecimiento logarítmico con n, aunque sí puedan variar los accesos a caché.
- **Actualización puntual:** $\Theta(\log n)$ en mejor y peor caso para $n \geq 2$. Después de escribir una hoja, el método `update` recalcula todos sus ancestros hasta la raíz. No termina anticipadamente si el nuevo valor coincide con el anterior o si un mínimo intermedio no cambia.
- **Memoria:** $\Theta(n)$ persistente y $\Theta(1)$ auxiliar por consulta y actualización. Los $2n$ objetos del vector representan $2ns$ bytes lógicos de datos, más los metadatos y cualquier capacidad adicional. La construcción usa $\Theta(1)$ espacio temporal fuera de ese almacenamiento, y el pico total es $\Theta(n)$. Las actualizaciones no crean nuevos nodos.

Estas cotas se refieren al código iterativo incorporado, no a otra implementación recursiva que reserve $4n$ posiciones. En ambas estructuras, el contenido de los valores no modifica las reservas principales ni la cantidad de pasos de construcción; el costo de consulta del árbol depende del rango, no de encontrar antes un valor especialmente pequeño.

2.3. Comparación de complejidades

La tabla resume RMQ con pares de tamaño fijo. La consulta excluye la construcción y el mantenimiento; en Sparse Table corresponde a `query_idempotent`. La columna de mantenimiento compara reconstruir Sparse Table con actualizar Segment Tree, no dos constructores.

Estructura	Construcción	Mejor consulta	Peor consulta	Mantenimiento	Memoria persistente
Sparse Table	$\Theta(n \log n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Segment Tree	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$

3. Implementación

Las dos implementaciones son clases genéricas de C++20 obtenidas del repositorio del curso y conservadas en `include/range_queries/`. Reciben un vector de elementos de tipo `T` y consultan intervalos semiabiertos $[i, j)$. Para obtener el índice del mínimo se utilizarán pares (valor, índice), cuyo orden lexicográfico resuelve empates por el menor índice. Se reemplazaron los comentarios y se añadió la cabecera `<bit>` a Sparse Table, sin modificar la lógica de los algoritmos.

3.1. Sparse Table

La clase `sparse_table` guarda una matriz de vectores `ST` y una función de combinación `f`. El constructor obtiene $m = \lfloor \log_2 n \rfloor$ mediante `_lg`, reserva $m+1$ filas de n elementos y copia la entrada en `ST[0]`. Dos ciclos construyen las filas siguientes: `p` representa la longitud de un bloque y `q` el doble; cada posición válida combina dos bloques consecutivos de la fila anterior.

Para RMQ se empleará `query_idempotent`, que calcula el nivel `b` a partir de la longitud del intervalo y combina los bloques de esa fila que comienzan en i y $j - 2^b$. La clase también incluye `query`, que recorre bloques disjuntos, pero no es la variante seleccionada para esta comparación. Como no dispone de `update`, en el escenario dinámico se reconstruirá desde el arreglo modificado. La implementación proviene del repositorio del curso [1].

3.2. Segment Tree

La clase `segment_tree` almacena la estructura en un vector `ST` de $2n$ elementos. El constructor copia la entrada en las hojas, desde `ST[n]` hasta `ST[2n - 1]`, y recorre los índices internos en orden descendente para guardar el mínimo de cada par de hijos mediante `_min`.

El método `query` desplaza los extremos del intervalo a las hojas e inicializa `q` con un elemento del rango. Un ciclo asciende por el árbol, incorpora los nodos de los bordes según la paridad de los extremos y conserva el menor candidato. El método `update` cambia una hoja y recalcula el mínimo de sus ancestros, combinando cada nodo con su hermano hasta alcanzar la raíz, sin reconstruir el vector. La implementación proviene del repositorio del curso [2].

4. Metodología experimental

4.1. Características de la máquina

Los experimentos se realizarán en una estación de trabajo con las siguientes características:

- **Procesador:** Intel Core i5-13420H (Raptor Lake), 8 núcleos y 12 hilos (4 P-Cores y 4 E-Cores), frecuencia base de 2.10 GHz y turbo de hasta 4.60 GHz. Cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB.
- **Memoria principal:** 16 GiB DDR (15 GiB utilizables por el sistema operativo).
- **Almacenamiento:** SSD NVMe WD PC SN740 de 512 GB, interfaz PCIe Gen4 $\times 4$.
- **Entorno de compilación:** Arch Linux x86_64 (kernel 6.x), GCC y las banderas `-std=c++20 -O3 -march=native`.
- **Afinidad de CPU:** se fijará el proceso de medición al núcleo lógico 2 mediante `taskset -c 2`. Esto reduce las migraciones entre núcleos, pero no elimina variaciones de frecuencia, temperatura ni la actividad concurrente del sistema.

4.2. Datos de entrada

Se generarán arreglos de **1.000, 4.000, 16.000, 64.000, 256.000 y 1.000.000 de enteros**, con valores uniformes en $[0, 10^9]$ y tres semillas fijas: **1729, 2718 y 31415**. Los auxiliares de `include/utils/rmq_inputs.hpp` adaptan las plantillas de los boletines 1 y 2, utilizando `mt19937_64` y muestreo por rechazo para evitar sesgo. Cada elemento será un par de enteros de 64 bits $(\text{valor}, \text{índice})$; el mínimo lexicográfico devolverá la posición más a la izquierda en caso de empate. Las semillas derivadas separan las secuencias de arreglos, consultas y modificaciones.

Se usarán longitudes $k \in \{1, 8, 63, 511, \lfloor n/2 \rfloor, n\}$. Para cada consulta se elegirá L uniformemente en $[0, n-k]$ y se fijará $R = L+k-1$, traduciendo $[L, R]$ a $[L, R+1)$ para invocar el código. El rango completo es un caso especial con una sola posición posible. En el escenario estático se prepararán **10.000 consultas por longitud**; en el dinámico, las posiciones de actualización serán uniformes en $[0, n-1]$, con reemplazo, y el nuevo valor pertenecerá a $[0, 10^9]$ y será distinto del vigente, conservando su índice. Ambas estructuras recibirán exactamente los mismos arreglos, cambios y consultas.

La generación y la conversión a pares quedarán fuera del cronometraje. Se registrarán configuración, semillas, versiones y huellas SHA-256 de fuentes, ejecutables y entradas. La corrección se comprobará exhaustivamente en arreglos pequeños, incluidos casos crecientes, decrecientes y con empates; en la batería se contrastarán hasta ocho consultas por longitud y estado contra un barrido lineal independiente, calculado antes de medir.

4.3. Experimentos e hipótesis

La ejecución se orquestará con scripts Bash basados en el boletín 3; Python se utilizará para registrar metadatos y generar tablas y figuras. Se realizarán **32 repeticiones por configuración y semilla**, después del calentamiento, siguiendo el esquema `uhr` de LELE con `std::chrono::steady_clock` y nanosegundos [9]. En cada repetición se medirá la construcción inicial y se reutilizarán las instancias primero para consultas estáticas y después para el recorrido dinámico. Cada método recorrerá su carga por separado, sin mantener viva la otra estructura; su orden se alternará entre repeticiones. Generación, validación, escritura, restauración y destrucción

quedarán fuera del reloj; la reconstrucción de Sparse Table medirá crear la nueva tabla, liberando la anterior después. Se consumirán las respuestas para evitar su eliminación por el compilador.

Los lotes estáticos se calibrarán fuera de muestra hasta **5 ms** o un máximo de **1.000.000 de consultas**, recorriendo cíclicamente los intervalos preparados y manteniendo el lote fijo entre repeticiones de cada método y configuración. Se registrará si el tope impide alcanzar el objetivo. El tiempo se dividirá por el número de operaciones efectivamente ejecutadas.

Cada recorrido dinámico tendrá **10 ciclos de una actualización seguida de 1.000 consultas**, repartidas equilibradamente entre las seis longitudes: 10 cambios y 10.000 consultas medidas por estructura y repetición. Cada estado actualizado servirá para todos los lotes, sin reconstruir por longitud ni repetir el cambio al medir consultas. No se forzará que los rangos incluyan la posición modificada; esta se verificará con una consulta unitaria fuera del reloj. Las repeticiones reiniciarán el mismo estado y secuencia; no se añadirán consultas dinámicas para calibrar.

Se conservarán CSV crudos y resúmenes con **media, desviación estándar muestral, mínimo, Q1, mediana, Q3 y máximo**, usando la convención de cuartiles de `uhr`. Los diez ciclos se agregarán dentro de cada repetición, normalizando por operaciones, no como muestras independientes. Cada semilla conservará su resumen; las figuras mostrarán media y una desviación estándar, **no intervalos de confianza**. Las 32 repeticiones caracterizan la variabilidad, pero no garantizan significancia estadística.

4.3.1. Construcción inicial en el escenario estático

- **Variable y controles:** variar n, con los mismos pares de entrada y tres semillas para ambas estructuras.
- **Resultado:** tiempo por construcción frente a n, sin cargar ese costo a las consultas de las instancias reutilizadas.

Hipótesis. Sparse Table tardará más al crecer n, por su preparación $\Theta(n \log n)$, frente a $\Theta(n)$ de Segment Tree. Ese costo adicional puede compensarse con suficientes consultas estáticas, pero no se presupone que ocurra para cualquier carga.

4.3.2. Consultas en el escenario estático

- **Variables y controles:** estudiar n con k fijo y k con n fijo, sin cambios en el arreglo y con consultas idénticas. Los rangos medio y completo se identificarán por separado, pues su longitud crece con n.
- **Resultado:** tiempo por consulta, comparando `query_idempotent` de Sparse Table con `query` de Segment Tree, sin incluir construcción.

Hipótesis. Sparse Table favorecerá las consultas por su costo $\Theta(1)$, frente al peor caso $\Theta(\log n)$ del árbol. No se espera esa diferencia en todos los rangos: los unitarios son constantes en ambas estructuras y mantener k fijo no obliga al árbol a crecer con n.

4.3.3. Mantenimiento en el escenario dinámico

- **Variable y controles:** variar n, reproduciendo los diez cambios y sus consultas, sin agrupar modificaciones antes de reconstruir Sparse Table.
- **Resultado:** tiempo por cambio: `update` del árbol frente a reconstrucción de la tabla, excluyendo consultas y construcción inicial.

Hipótesis. Segment Tree tendrá menor tiempo de mantenimiento por su actualización $\Theta(\log n)$, frente a reconstruir Sparse Table en $\Theta(n \log n)$; la diferencia debería crecer con n. Se comparan estrategias de mantenimiento, no dos actualizaciones nativas.

4.3.4. Consultas en el escenario dinámico

- **Variables y controles:** estudiar n y k sobre los mismos estados modificados, usando los lotes del recorrido anterior con ambas estructuras ya actualizadas.
- **Resultado:** tiempo por consulta después de cada cambio, separado del mantenimiento y sin duplicar reconstrucciones.

Hipótesis. Sparse Table conservará su consulta $\Theta(1)$ tras reconstruirse; la ventaja dinámica de Segment Tree corresponde al menor mantenimiento, no necesariamente a consultar más rápido. El estado de caché puede alterar tiempos respecto del escenario estático, pero no estas cotas.

4.3.5. Análisis del espacio utilizado

En una pasada independiente se registrarán bytes retenidos y picos de asignación, sin activar el seguimiento durante el cronometraje ni incluir la entrada externa o metadatos del asignador. Se distinguirán nuevas reservas y memoria previa al reconstruir; no son medidas de memoria residente total. **Hipótesis.** Sparse Table ocupará más espacio, por sus $n(\lfloor \log_2 n \rfloor + 1)$ pares frente a los $2n$ del árbol. Las actualizaciones reutilizan el vector, mientras que reconstruir exige nuevas reservas y puede elevar el pico al coexistir ambas versiones. Este análisis acompaña los cuatro experimentos, sin añadir otro de tiempo.

Referencias

[1] LELE. *Generic Sparse Table* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/sparse_table.cpp.

[2] LELE. *Generic Segment Tree for RMQs* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/segment_tree.cpp.

[3] Colaboradores de CP-Algorithms. *Sparse Table*. https://cp-algorithms.com/data_structures/sparse-table.html.

[4] Colaboradores de CP-Algorithms. *Segment Tree*. https://cp-algorithms.com/data_structures/segment_tree.html.

[5] AtCoder Library. *Segtree*: documentación de árboles de segmentos sobre monoides. https://atcoder.github.io/ac-library/production/document_en/segtree.html.

[6] Cppreference. *std::bit_width*. https://en.cppreference.com/w/cpp/numeric/bit_width.html.

[7] Colaboradores de CP-Algorithms. *Euclidean algorithm for computing the greatest common divisor*. https://cp-algorithms.com/algebra/euclid-algorithm.html.

[8] Cppreference. *std::lcm*. https://en.cppreference.com/w/cpp/numeric/lcm.html.

[9] Lovera, L. (LELE). (s. f.). *uhr: generic time performance tester* [Código fuente]. Plantilla de medición de la ayudantía, disponible en `utils/uhr.cpp` del repositorio de estos boletines. Referencia del autor utilizada en los boletines anteriores: https://github.com/leonardlover.